

ネットワーク系演習 I

- システムソフトウェア演習 -

1 目的

簡単なプログラミング言語のコンパイラを作成することにより、コンパイラの構造、プログラミング言語、および計算機アーキテクチャについて理解を深めると共に、ある程度大きなプログラム作成を通じてソフトウェアシステム構築技術を習得する。

2 コンパイラの基礎

C 言語や Pascal などの高級言語は、人間に分かりやすい書き方でプログラムを書ける計算機言語である。しかし、そのままでは計算機が実行できる機械語ではないため、機械語に変換する操作が必要である。この変換を行うプログラムをコンパイラという。コンパイラに入力する高級言語で書かれたプログラムを原始プログラム、コンパイラが出力する機械語あるいはそれに近い低レベルの言語で書かれたプログラムを目的プログラム、目的プログラムを最終的に実行させる計算機を目的計算機と呼ぶ。一般に計算機にはアセンブラが存在するので、コンパイラの目的プログラム言語をアセンブラ言語とし、アセンブラが実際の機械語に変換することが多い。本実験で作成するコンパイラも、目的プログラム言語をアセンブラ言語とし、アセンブラ言語のプログラムから機械語プログラムへの変換は、こちらで用意するアセンブラで行う。

コンパイラはいくつかのフェーズからなる。フェーズとは処理の論理的な一つのまとまりであり、原始プログラムからフェーズ毎に徐々に目的プログラムへ近づいていくと考えることができる。単純なコンパイラの典型的なフェーズの構成を図 1 に示す。四角の中が各フェーズである。以下、簡単に各フェーズの説明を行う。

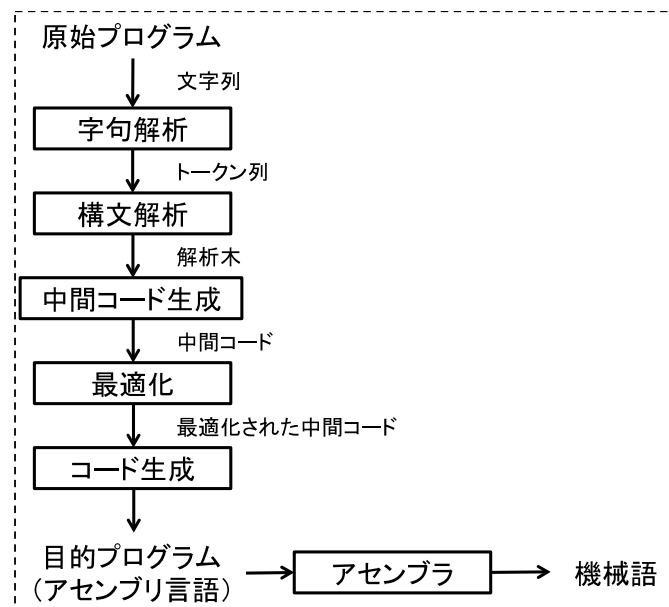


図 1 コンパイラの構成

2.1 字句解析

原始プログラムは人間が見ると、いくつかの意味のある最小の単位（文字がいくつか集まったもの）から構成されていることが自然と分かる。例えば、図 2 のような C 言語のプログラムを考

える。文字がいくつか集まった最小の単位は、「main」、「(」、「)」、「{」、「int」、「i」、「,」...などである。人間であれば容易にこの最小単位を認識できるが、計算機が読み込む原始プログラムは計算機から見ると単なる文字を表す数字の列であり、工夫をしなければ自然とこの最小単位に分解してくれる訳ではない。この最小単位に分ける処理を字句解析と呼び、主に次のような 2 つの処理を行う。

- ・最小単位の区切りを解析する。

例えば、「sum3 = 0;」という文字列に関して、「sum3」と「=」と「0」と「;」で区切れるということを解析する。

- ・最小単位が表している意味を解析する。

例えば、「sum3 = 0;」という文字列に関して、「sum3」は変数名や関数名を表す文字列、「=」は代入記号、「0」は数字、「;」は 1 つの文の終わりを表す記号であることを解析する。変数名についている数字の「3」は、あくまでも変数名などの名前の一部であり、「0」のような数字とは異なることを解析する。

本実験ではこちらで用意した字句解析プログラムを使用する。この字句解析プログラムは原始プログラムを読み込んで、自動的に意味のある最小単位を順番に切り出し、その最小単位の情報を持つトークンと呼ばれるデータ構造の列に変換する(実際には一回呼び出すごとに 1 つの最小単位を切り出し、1 つのトークンを返す)。この字句解析プログラムの詳細は付録 C に付ける。

```
main() {
    int i, sum3;
    sum3 = 0;
    for (i = 0; i < 10; i++) {
        sum3 = sum3 + i;
    }
}
```

図 2 C 言語のプログラム

2.2 構文解析・中間コード生成

上記の字句解析プログラムで最小単位に分解された原始プログラムは、構文解析によって意味のある単位に階層的にまとめられる。例えば、図 2 の例のプログラムで構文解析された大まかな結果を見てみると、図 3 のように、全体が main と呼ばれる関数であり、その中で変数の宣言部、代入文、for 文などによって階層的に構成されていることが分かる。for 文は、さらに代入文、条件文などから構成されている。さらに、一つの代入文は図 4 のように階層的に構成されている。

このように、一つの意味のある大きな単位に最小単位から階層的にまとめるフェーズを、構文解析と呼ぶ。構文解析の結果から単にプログラムの構造が分かるだけでなく、まとめた単位がその単位を含むより大きな単位の中でどのような意味をもっているのかについても、知ることができる。例えば図 4 の例では、全体が「代入文」であることから、その一部である「数式」の部分の計算結果を「変数」の中に代入する目的プログラムを出力するように、コンパイラを作ることができる。さらに、「数式」と解析された部分に関しては、「sum3」という「変数」の値と「i」という「変数」の値を加算するような、目的プログラムを出力すればよい。このよ

うに、字句解析で得られた最小単位の役割が構文解析によって解析され、その役割に応じたコンパイラの内部ルーチンによって、目的プログラム、あるいは中間コードが生成される。中間コードとは、目的プログラムに変換する前に最適化などを簡単に行うために、構文解析の結果などの情報を保持している中間的なコードである。最適化などを行わない場合は、構文解析フェーズから直接、目的プログラムを生成することもある。本実験では、数式に関しては中間コードを用いるが、その他では構文解析フェーズから直接目的プログラムを生成する。

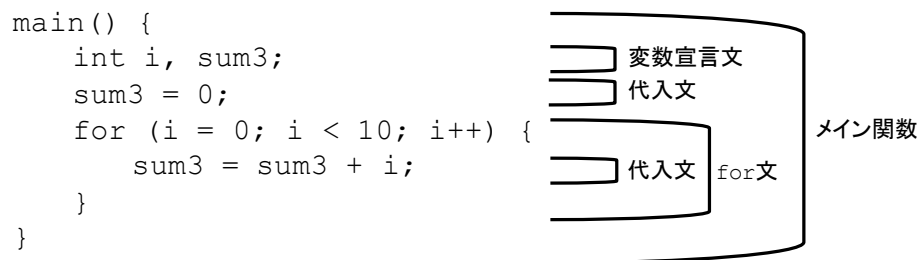


図 3 C 言語のプログラムの構造

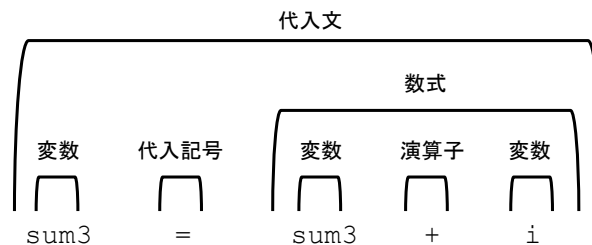


図 4 代入文の構造

2.3 最適化・コード生成

中間コードからコード生成を直接行い、目的プログラムを生成してもかまわないが、構文解析・中間コード生成フェーズからの出力は効率的でない場合が多い。このため、目的プログラムをより速く実行できるように、必要でない部分を削ったり、実行順序の入れ替えなどを行う。これが最適化のフェーズである。本実験では特にこの部分を作成することを義務とはしないが、興味がある学生は作成した方がよい。

最適化が終わった中間コードを目的プログラムに変換するフェーズを、コード生成と呼ぶ。これは、機械的に目的プログラムに置き換えることによってなされる。構文解析の説明で述べたように、本実験では数式以外の部分は構文解析と同時に目的プログラムを生成するので、コード生成のフェーズは数式の部分のコンパイルを行う場合のみである。また、最適化も行わないのであれば、数式の部分も中間コードを生成せず、直接、目的プログラムを生成してもよい。

3 各作成実験の解説

3.1 作成実験の概要

3.1.1 作成するコンパイラの概要

4 段階の作成実験を通して、最終的に次のようなコンパイラを生成する。

原始プログラムの言語(構文・意味) 原始プログラムの言語に関しては、作成実験の段階によって、徐々により複雑な入力言語を処理できるように、拡張していく方法を取る。このため、各作成実験に応じた原始プログラムの構文、意味が与えられる。おおよそ、PL/0 と呼ばれる Pascal に似た言語のサブセット+ α である。最終的には次のような言語を受理するコンパイラとなる。

- ・ PASCAL のサブセット(PL/0)+ α
- ・ データ型は整数型のみ
- ・ 再帰呼び出し可能とする
- ・ 手続き定義のネストはない
- ・ 制御文は if 文、while 文、複合文のみ
- ・ 手続きの引数は、値渡し

目的計算機の仕様の概略 目的計算機のアーキテクチャ、および目的プログラムの言語は全ての段階の作成実験で共通であるが、使用する目的プログラムの命令は徐々に多くなっていく。詳しくは付録 A、付録 B にそれぞれ解説している。概略は次の通りである。

- ・ レジスタマシン
 - 汎用レジスタ 4 個
 - ベースレジスタ 1 個
 - スタックポインタ 1 個
 - 基本命令 15 種

コンパイラ記述言語 コンパイラ記述言語とは、コンパイラを作るために使用する言語のことである。本実験では原則的に C 言語を考えており、C 言語で作成するためにいくつかのサンプルプログラムなどを用意してあるが、他の言語、例えば LISP などの言語でも作成してもよい。しかしその場合、サンプルプログラムなどのサポートは完全に行われないので、自分の責任ですべてを作らなければならない。

3.1.2 実験スケジュール

作成実験 1: 「簡単なコンパイラの作成と実験ツール」(第 1 週) 非常に簡単な言語のコンパイラを作成し、実験のためのツールに慣れ、コンパイラの概要、ターゲットマシンと命令セット、字句解析、コード生成についての大まかな知識を修得する。

作成実験 2: 「変数の扱いと if 文, while 文」(第 2 週～第 3 週) 変数、if 文、while 文を含む言語のコンパイラへと、作成実験 1 で作成したコンパイラを拡張する。

作成実験 3: 「数式の翻訳と構文解析」(第 4 週～第 5 週) 一般的な数式を含む言語のコンパイラへの拡張を行う。

作成実験 4: 「実行モデルと手続き」(第 6 週～第 7 週) 手続き定義と手続き呼び出しができる

言語のコンパイラへの拡張を行う。

3.1.3 実験グループ

コンパイラは1人で1つ作成する。グループでの開発は認めない。

3.2 作成実験 1 「簡単なコンパイラの作成と実験ツール」

この段階では非常に簡単な構文を持つ言語のコンパイラを作成し、コンパイラの概要、実験ツールの使用法の修得をめざす。

3.2.1 原始プログラムの構文と意味論

作成実験 1 で作成するコンパイラの原始プログラムの例を図 5、構文図を図 6 に示す。構文図は、コンパイラが目的プログラムに変換できる原始プログラムの「形」を正確に規定する。日本語で言えば日本語文法にあたるが、計算機の言語は自然言語のような曖昧さは許さない。構文図で規定された形以外のプログラムは、文法誤りとしてリジェクトされる。また原始プログラムの規定としてもう一つ重要なものに、そのプログラムが意味することを規定する「意味論」がある。例えば、「 $r = 3 * x$ 」という式があった場合、C 言語のプログラマならば、「3」と「 x 」という変数の値をかけたものを「 r 」という変数に格納すると考えるが、計算機言語を全く知らない人は「 $*$ 」の記号を乗算だとは思わないかもしれない。

『「 $*$ 」は乗算の記号である』という規定が意味論である。しかし、意味論を正確に記述することは困難であるので、本実験では本指導書の中で注意するところだけ日本語で与える。特に、規定されていない部分は常識的に解釈して欲しい。すなわち、「 $*$ 」はかけ算の演算子であり、わり算の演算子ではない。

図 6 の構文図の意味論は次の日本語文で規定される。特に規定されていない部分は常識的に解釈すること(この場合は、特に何もしないということになる)。

図 6 の意味論：「数式(expression)を計算し、数式が現れる順番にその結果をディスプレイに表示する。表示する結果は各数式の結果ごとに改行を行い、1 行に 1 つの結果を表示する。」

すなわち、図 5 の例 3 のプログラムの実行結果は図 7 のようになる。

例1: <pre>program example1; 526 * 321.</pre> 例2: <pre>program example2; begin 100 + 21; 30 - 643; 10 * 32 end.</pre>	例3: <pre>program example3; begin begin 200 + 10; 30 - 8 end; 13 * 46 end.</pre>
---	--

図 5 作成実験 1 の原始プログラム例

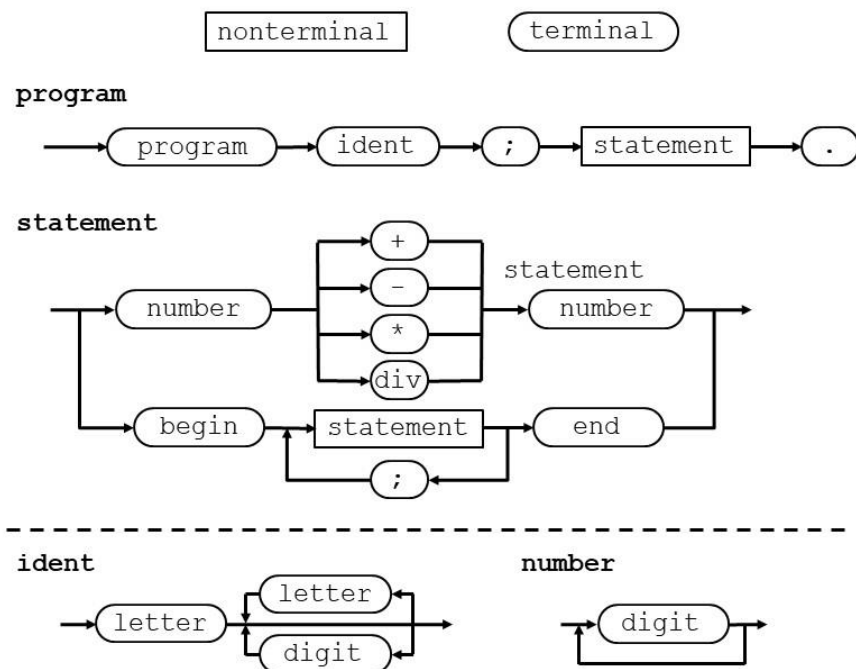


図 6 作成実験 1 の構文図

```

210
22
598

```

図 7 プログラムの出力例

3.2.2 実験ツール

2つの実験ツールをこのプロジェクト実験を通して使用する。機械語シミュレータとアセンブラである。機械語シミュレータは、本実験で仮定している計算機の機械語を、実験で用いるコンピュータ上でシミュレーションするために使用する。実験で用いる実際の計算機の機械語は非常に複雑であり、それを学習するだけで時間がかかるため、このような方法を取る。アセンブラは本実験で作成するコンパイラが出力する目的プログラム(アセンブラ言語で書かれている)をアセンブルして、上記の機械語シミュレータで実行できる機械語に変換するために使用する。実際に諸君が作成するコンパイラを用いて、原始プログラムを実行するまでの手順を図8に示す。アセンブラと機械語シミュレータの詳しい使い方は、付録A、付録Bにそれぞれ解説する。

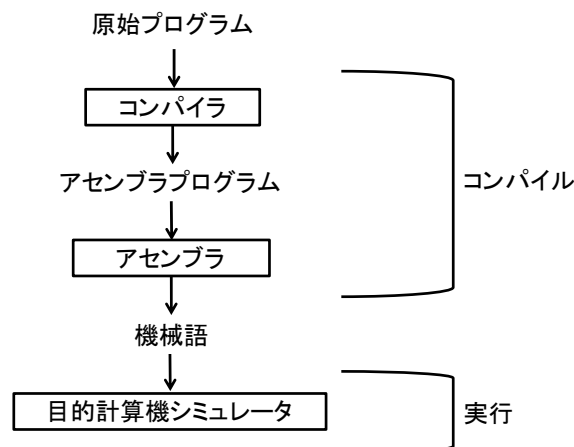


図 8 コンパイルと実行の手順

3.2.3 実際の作成

付録Eに、**statement**を除いた部分の構文解析だけを行うC言語のプログラムを示す。このプログラムでは字句解析に、付録Cに詳しく説明するルーチンを用いている。字句解析ルーチンから得られた1つのトークンを、C言語の**if**文で判断することにより、図6の構文図をたどっていくように作られている。構文図で可能性がないトークンが来たら、エラーを出力するようになっている。構文図の独立した各部分をC言語の関数で記述することによって、「**begin...end**」のネストのような再帰的な構文も処理できる。付録Eを参考に**statement**を解析するルーチンを追加すれば、構文図をたどり、構文解析するルーチンができあがる。さらに、構文の意味に従い、アセンブラプログラムを出力するように改良すれば、簡単なコンパイラができる。

作成実験1の意味論では、「数式」すなわち **expression** にだけ意味があるから、**expression** を構文解析すると同時に、「計算を行い、結果をディスプレイに表示するアセンブラプログラム」を出力すればよい。例えば、図9のような変換がなされればよい。ここで、出力のアセンブラプログラムの各部分は図9に示したように、原始プログラムの数式に対応している。原始プログラムに数式が複数出てくる場合は、数式を解析するコンパイラのルーチンを複数回通る。したがって、数式を解析するルーチンは、一つの数式のアセンブラプログラムを正確に出力するように作成すればよい。

数式を解析するルーチンは、構文図に従えば、数字(**number**)を見て、次に演算子を見、最後にまた数字を見るはずである。アセンブラプログラムの方では、まず1つ目の数字をあるレジス

タにロード(**loadi**)し、次にそのレジスタと最後の数字を演算子が意味する演算で処理(**addi** など)すればよい。このように 2 つの命令で構成されるが、2 つ目の命令は演算子と最後の数字を見た後でしか出力できない。このため、演算子を覚えておいて、最後の数字を見た時点で 2 つ目の演算を行う命令を出力する。最後に、原始プログラムの意味に従って、計算結果が格納されているレジスタの内容を **writed** 命令で出力し、**loadi**、**writed** 命令で改行するようなアセンブラプログラムを出力すればよい。

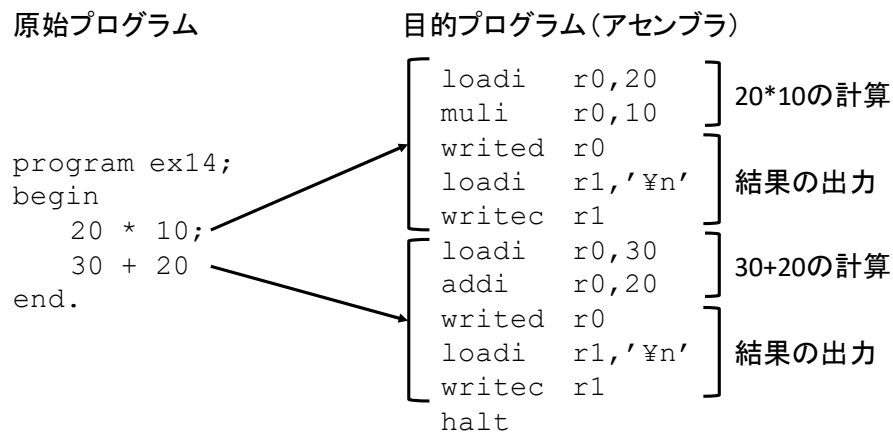


図 9 作成実験 1 のコンパイル例

3.3 作成実験 2 「変数の扱いと if 文, while 文」

3.3.1 原始プログラムの構文と意味論

作成実験 2 では、変数、if 文、while 文、write 文を扱えるように、作成実験 1 のコンパイラを拡張する。作成実験 2 で作成するコンパイラの原始プログラムの例を図 10 に、構文図を図 11 に示す。ただし、図 11 の点線で囲まれている部分は関係ない(作成実験 4 で実現する)。write 文の導入に伴い、数式の意味としてディスプレイに表示することは省かれ、数式はその値を計算するという意味だけになる。write 文の意味は、「write 文の後ろに続く変数の値をスペース 1 個で区切りながら 1 行に表示し、最後に改行コードを出力する」である。また、変数を使用する場合は、構文図の **outblock** の中で宣言しておかなければならないとする。

3.3.2 変数の処理

変数は原始プログラム中で数値を記憶するために使用される。このため、目的プログラムでは、原始プログラムでの変数に相当する記憶場所を目的計算機内のメモリに確保し、そこを使用しなければならない。機械語では記憶場所はメモリのアドレスで管理されるため、それぞれの変数がどのアドレスに対応するかをコンパイラが決定し、その決定に従って、目的プログラムを生成する必要がある。この変数とアドレスの対応を表現する表を記号表と呼ぶ。

例1: <pre> program ex21; var x, y; begin x := 10; y := 2 * x; write x, y end. </pre> 例2: <pre> program ex22; var x, y, z; begin x := 100; y := 200; if y - x >= 50 then z := 2 * x else z := 3 * x; write x, y, z end. </pre>	例3: <pre> program ex23; var i, sum; begin i := 0; sum := 0; while i <= 10 do begin sum := sum + i; i := i + 1 end; write sum end. </pre>
---	--

図 10 作成実験 2 の原始プログラム例

構文図と意味論によれば、使用する変数を必ず原始プログラムの最初の段階で宣言しなければならないため、コンパイラにとっては都合がよい。すなわち、原始プログラムを読み込む最初の段階(**outblock**)で、そのプログラムが使用する全変数を知ることができるため、この段階で変数表を作成できる。プログラム本体(**statement**)のコンパイルでは、この記号表を参照しながら、変数を機械語のアドレスに変換すればよい。

実際の変数の記憶場所として、目的計算機のデータ領域を使用する。目的計算機の命令体系(付録 A 参照)によれば、**type4-1** のレジスターメモリ間の演算・転送命令を実行するとき、メモリアドレスに自動的にデータセグメントレジスタの値が加算される。このため、データ領域は 0 番地から始まっていると考えることができる。また、データセグメントレジスタの値は十分に大きな値が設定されているため、機械語プログラムの格納されているプログラム領域とは、独立であると仮定してよい。

結局、記号表を作る際のアドレスの決定は、最初に **outblock** で変数が宣言される際の順番に従って、**0,1,2,...** と付けていけばよい。記号表は変数とアドレスのペアのリストとなる。例えば、図 12 で表されるような構造となる。原始プログラムに変数が現れたら、この記号表の変数名の部分を探索して、一致する変数名に対応するアドレスを記憶場所として使用する。変数の使われ方は、代入文の代入記号「:=」の左右で異なる。左にある場合は、右で計算された結果をその変数に対応するアドレスへ格納(**store**)することになり、右にある場合は、その変数に対応するアドレスの内容を計算に使用するため、**load** 命令かあるいはレジスターメモリ間の演算命令が使用される。

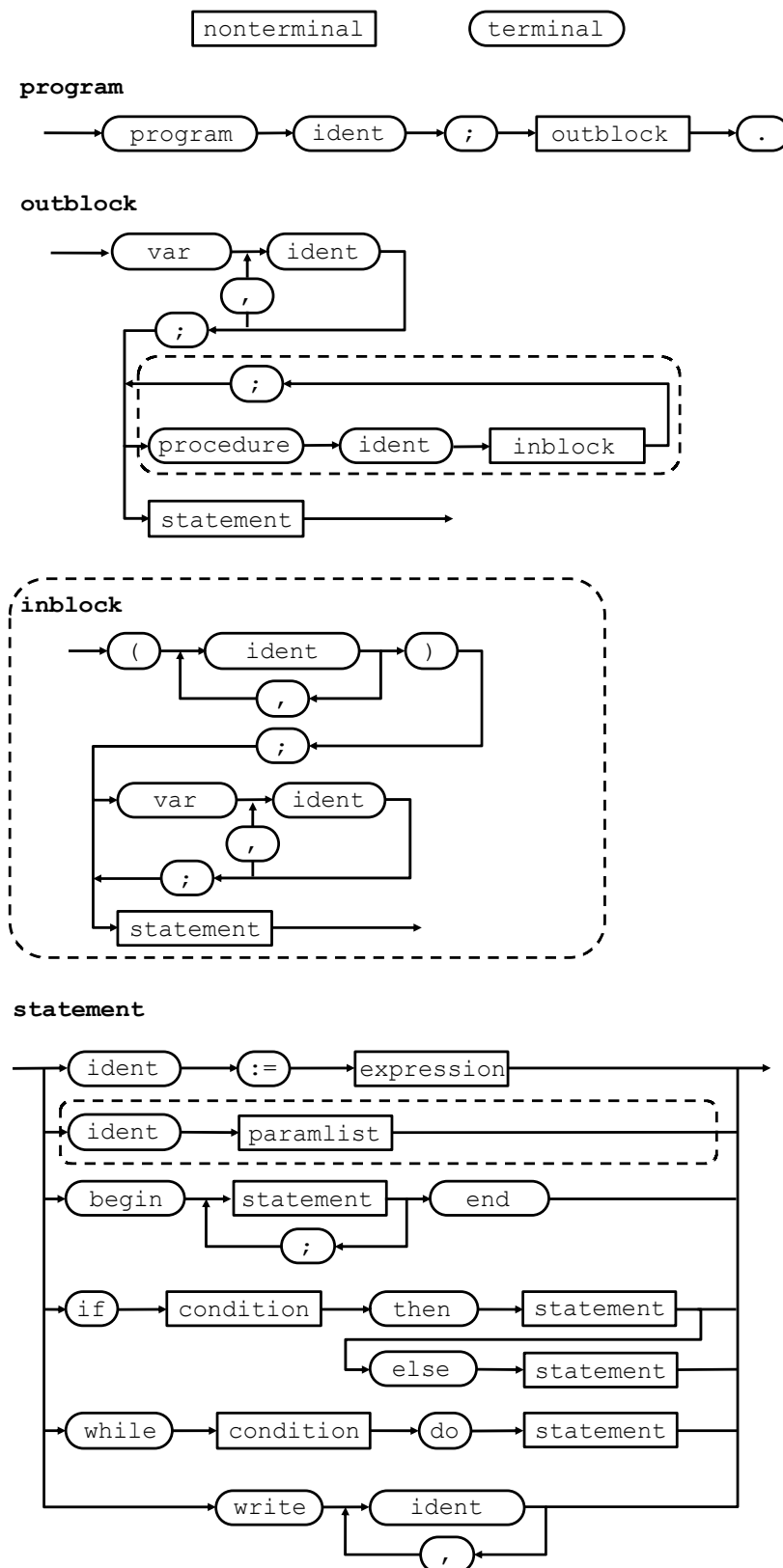


図 11 作成実験 2,3,4 の構文図 (その 1)

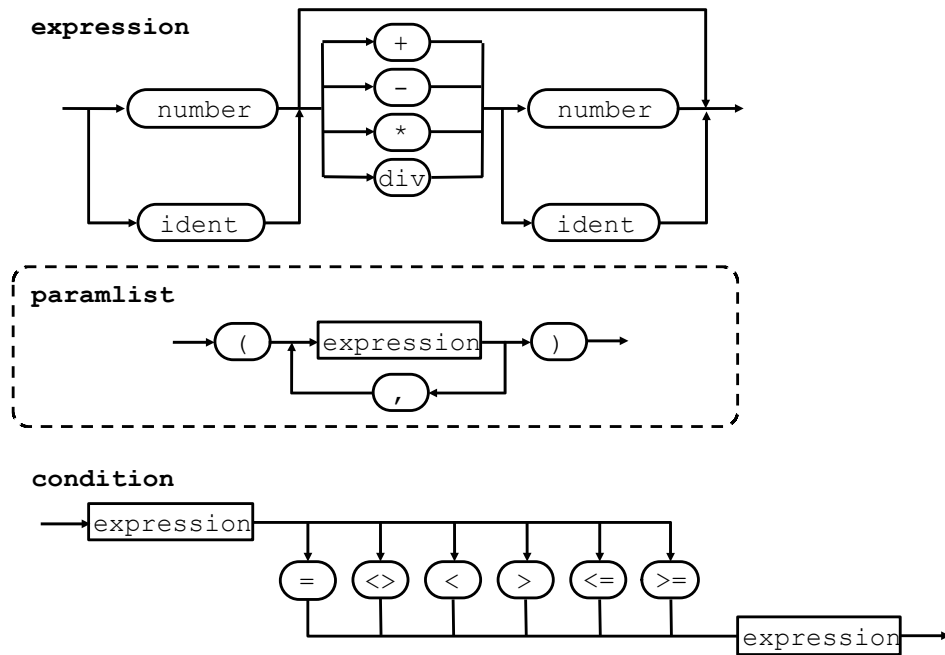


図 11 作成実験 2,3,4 の構文図 (その 2)

変数名	アドレス
x	0
y	1
z	2
i	3
sum	4
.	.
.	.
.	.

図 12 記号表の例

3.3.3 if 文, while 文

if 文と while 文に対応する目的プログラムは、図 13 のような構成となる。if 文の then 部、else 部、while 文の本体はすべて statement であり、statement をコンパイルするルーチンにまかせればよい。条件文のコンパイルと、その条件によってどこへジャンプするかを目的プログラムに反映させることが、主な作業である。

条件文は 2 つの expression と大小を表す記号からなっている。expression の値を計算する目的プログラムはすでに作成した expression をコンパイルするルーチンがやってくれる。注意するのは、目的プログラム中で 1 つ目の expression を計算した結果をどこかにしまっておかなければならないことである。例えば、expression を計算するプログラムで使用するレジスタに 1 つ目の expression の計算結果を保存していれば、2 つ目の expression を計算する部分で 1 つ目の結果を破壊してしまう。1 つ目の expression を計算した結果は、計算に使用しないレジスタ、あるいはメモリ上に格納するようにすればよい。

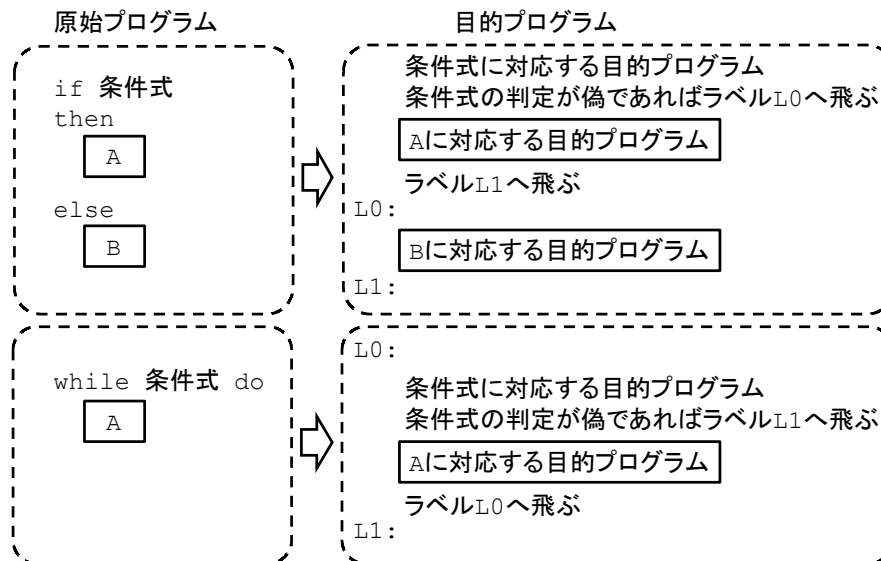


図 13 if 文と while 文の目的プログラムの構成

3.4 作成実験 3

3.4.1 原始プログラムの構文と意味論

作成実験 3 では、数式(expression)を一般的な式へ拡張する。原始プログラムの例を図 14 に、作成実験 2 と異なる構文図の部分(expression のみ)を図 15 に、それぞれ示す。作成実験 1,2 では、単なる加減乗除を 2 つの数値または変数に対して行うことができるのみであったが、作成実験 3 では加減乗除の組み合わせや括弧の使用を許した数式を扱う。当然、乗除の演算が加減の演算よりも優先され、さらに括弧の中の計算が優先される。計算機の機械語では一般に、加減乗除を 1 つの命令で 1 種類しか行えないので、構造を持った数式のどの部分とどの部分をどのような順序で計算すればよいかを、正確に解析しなければならない。

本作成実験では、数式の解析を行う 2 種類の方法を作成する。1 つは演算子順位構文解析法と呼ばれるもので、もう 1 つは expression 以外の部分の解析で使用している構文図をそのまま C 言語の関数で実現する方法である。概念的には、前者はボトムアップ構文解析、後者はトップダウン構文解析と呼ばれる解析法の仲間(前者はやや特殊)である。

また、数式を処理する場合はやや複雑な処理を行うため、直接アセンブラ言語にコンパイルするのではなく、中間的な言語にまず変換してから、最後にまとめてアセンブラ言語に変換する手法が使われる。中間的な言語で書かれたプログラムを中間コードと呼び、本実験では「4 つ組」というものを利用する。以下、演算子順位構文解析法とトップダウン構文解析法について簡単に説明する。

3.4.2 演算子順位構文解析

数式のどの部分から先に計算すればよいかは、数式の構造を書けばよく分かる。図 16 は数式の構造の例である。数式全体の値は各部分構造の結果を演算して得られるため、構造の内側のものから順番に計算していけばよい。数式を目的プログラムに変換するためには、この構造を解析し、一番内側のものから順番に目的コードを出力する。

例1:

```

program pi;
var i, rand, ca, cm, pi, qpi, x, y;
begin
  i := 0;
  qpi := 0;
  rand := 3827;
  ca := 8358;
  cm := 3797;
  while i < 1000 do begin
    rand := (rand + ca) * cm - (((rand + ca) * cm) div 100000) * 100000;
    rand := rand div 10;
    x := rand div 10;
    rand := (rand + ca) * cm - (((rand + ca) * cm) div 100000) * 100000;
    rand := rand div 10;
    y := rand div 10;
    if x * x + y * y < 1000000 then
      qpi := qpi + 1;
    i := i + 1
  end;
  pi := qpi * 4;
  write pi
end.

```

図 14 作成実験 3 の原始プログラム例

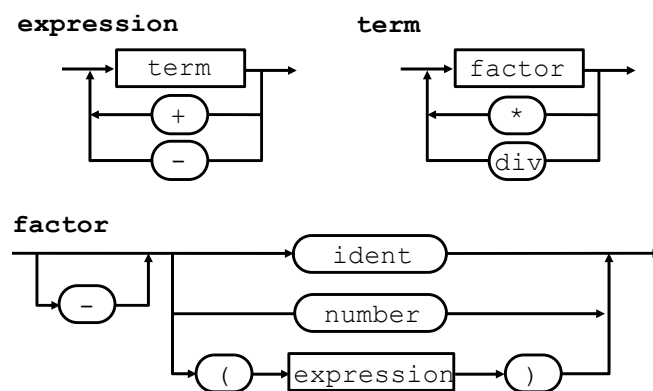


図 15 作成実験 3 の構文図 (変更分)

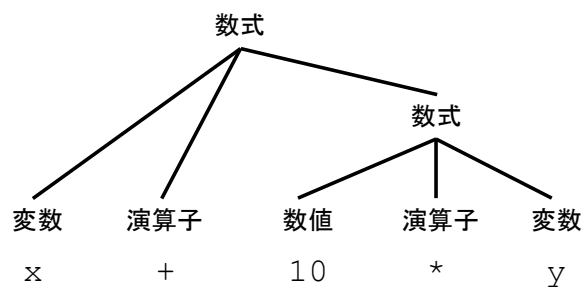


図 16 数式の構造

出力される目的プログラムは、部分構造ごとに計算結果がメモリ上のある場所に格納され、その部分構造は変数と同じように扱えばよい。すなわち、ある部分構造の結果を使って(優先順位が低い)より外側の部分が計算される場合は、内側の部分を変数と見なしてその結果が格納されているアドレスを用いて計算するような、プログラムを出力する。

一つの部分構造を計算するプログラムを「4 組」と呼ばれる中間コードで表現し、先に計算すべきものから順に並べたものが図 17 である。それぞれの 4 組は、4 つの記号の組から成っている。4 組を(a1, a2, a3, a4)で表せば、a1 が演算子、a2 a3 が被演算子、a4 が結果の格納場所を表す。例えば、(*, x, 10, T1)という 4 組は、x という変数と 10 という数値をかけて、T1 という場所に格納するということを意味している。格納場所は数式には明示的に書いてないため、コンパイラが決定しなければならない。各部分構造に 1 つずつ異なる格納場所を割り当てれば容易に決定できる。

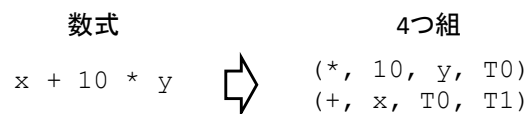


図 17 4 組の例

構文の解析は若干複雑なため、このような中間コードを仮定することにより、全体の処理を見通しのよいものにできる。以下の説明では 4 組の出力を最終目的とし、4 組から実際の目的プログラムへの変換については述べない。

構造を解析するために数式を左から右に見ていき、スタックに今すぐに使わない演算子、被演算子を積み、スタックへの積み下ろしを演算子の優先順位で決定していく解析手法を、演算子順位構文解析と呼ぶ。スタックとは、要素を一度に 1 つずつ入れたり・出したりできるデータ構造で、入れた順番と逆順に(すなわち、最初に入れたものが最後に)出てくるものである。数式処理における優先順位の低い演算子は最初に出てきても後で使われるため、このデータ構造は非常に都合がよい。

直感的には、数式のある部分式に対応するプログラムを出力するタイミングは、その部分式が右隣の演算子の優先順位より高い場合である。もし、右隣の演算子の方が左隣よりも優先順位が高い場合は、その部分式に対応するプログラムは、右隣のより優先順位の高い部分式の計算プログラムを出力した後になる。このため、右隣の優先順位が高い場合にはどんどんスタックに入れていき、右隣の処理が済み、右隣の優先順位が低くなった時点でスタックから取り出し、その部分の計算プログラムを出力する。例えば、次のような数式の処理を考える。ここで、最初と最後の「\$」は数式の始まりと終わりを表す記号である。

\$ x + 10 * y \$

この式を左から右に見ていき、10 のところまで来た時点では、「x + 10」の 4 組を出力できない。なぜなら、その後どのような演算子が来るか分からないからである。次に「+」や「-」のように左の演算子よりも優先順位が同じか低いものが来た場合は出力できる。この場合は次に「*」が来るので出力できない。出力されないのが、すべての項がスタックに積まれている。さらに y が来るが、これは演算子ではないので、やはりそのままスタックに積まれる。ここまでの様子を図 18(a)に示す。最後に数式の終わりを表す「\$」が来てはじめて、スタックから順番に部分式が取り出され、4 組が出力される。最初に「10 * y」の部分を取り出され、4

つ組が出力される。1つ目の4つ組が取り出された後の様子を図18(b)に示す。上で述べたように、数式の部分式を処理した後、その部分式は変数と同じように扱われる。したがって、部分式の格納場所である番地 T1 が、取り出された部分式の代わりにスタックのトップに置かれている。さらに、次の部分式を処理した結果を図18(c)に示す。スタックには数式の最初を表す「\$」と全体の計算結果が格納されているアドレスのみが入っており、現在見ている数式の部分が数式の終わりを表す「\$」であることから、処理が終了したことが分かる。結果として、正しい4つ組が得られている。

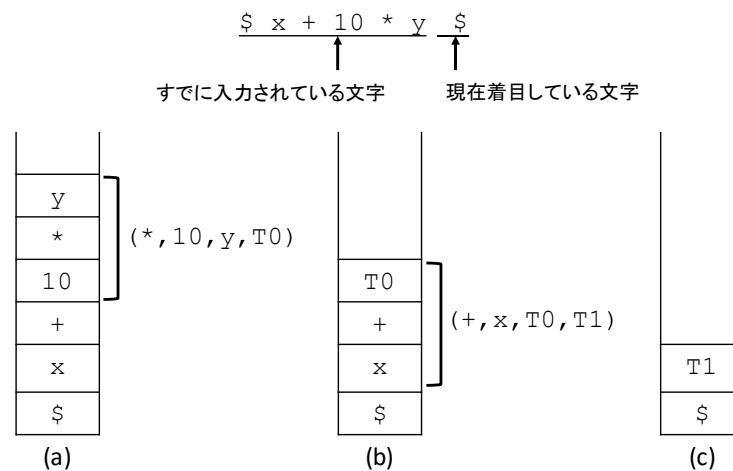


図 18 演算子順位法のスタックの変化

ここまでの説明は非常に直感的なものである。詳しいアルゴリズムと各演算子の優先順位の決め方を付録Dにつけるので参照せよ。

3.4.3 トップダウン構文解析

作成実験1,2で、構文図の **program** や **statement** などの部分を処理した手法は、トップダウン構文解析手法の1つである。そこでは、トークンを1つ読むだけでは構文図のどこに行くかを決定し、次の構文が他の構文を呼んでいる場合は、次の構文を処理するためのルーチンを呼ぶことによって解析した。数式も **program** や **statement** の部分と同じような構文図(図15)で表現できるため、同じ手法によって処理できる。

足し算・引き算を表現する **expression** 構文の中で、再帰的にかけ算・わり算を表現する **term** が呼ばれている。各構文をC言語の関数で表現すれば、**expression** を処理する関数が最後に実際の目的プログラムを出力する場合、すでに **term** の処理は終わっている。なぜなら、**term** の処理が終わらなければ、**term** を呼び出した **expression** へ実行は戻ってこないからである。こうして、常にかけ算・わり算を計算する目的プログラムが、足し算・引き算を計算する目的プログラムよりも先に出力される。C言語の関数は再帰呼び出しができるために、構文が再帰的に書かれている場合は大丈夫である。これが直感的な仕組みである。

このような方法で作成すれば、演算子順位法と比べて非常に簡単に、数式をコンパイルするルーチンを作成することができる。簡単に作成できる理由は、C言語の再帰呼び出しにある。再帰呼び出しには必ずスタックが使われているが、このスタックはC言語が自動的に用意してくれるものである。この自動的に用意されたスタックを、この手法はうまく利用している。演算子

順位法ではスタックをみずから作って操作していたために、複雑になったと考えることができる。

3.5 作成実験 4

作成実験 4 では再帰呼び出し可能な手続きを含む言語を扱えるように作成実験 3 のコンパイラを拡張する。

例1:

```
program hanoi;
var n;

procedure hanoi(a, b, c, n);
var i;
if n = 1 then write a, b
else
begin
    hanoi(a, c, b, n-1);
    write a, b;
    hanoi(c, b, a, n-1)
end;

begin
    n := 5;
    hanoi(1, 2, 3, n)
end.
```

図 19 作成実験 4 の原始プログラム例

3.5.1 原始プログラムの構文と意味論

図 11 の **expression** を図 15 のもので置き換え、さらに点線の内部も含むものが、作成実験 4 で作成するコンパイラの原始プログラム言語の構文である。図 19 に原始プログラムの例を示す。拡張は手続きが導入され、手続き定義、手続き呼び出しが行えることである。手続きは再帰呼び出し可能とし、手続きの中で手続きは定義できないものとする。引数は値渡しであり、手続きの返り値はないので、手続きの処理結果は大域変数を介して行われる。

再帰呼び出し可能とするためには、目的プログラムがどのように実行されるかをよく検討する必要がある。目的プログラムが実行されときのモデルを実行時環境と呼ぶ。次の節では、再帰呼び出しを可能とする一つの実行時環境を説明する。

3.5.2 実行時環境

プログラムの実行時の振る舞いは動的であり、原始プログラムの手続きに現れる一つの局所変数が、実行時には手続きの再帰呼び出しによって、複数の実体に対応する場合がある。これを処理するために、手続きが呼ばれるたびに新しい変数領域を局所変数のために確保する必要がある。その他、手続きは呼ばれるときに引数を与えなければならないし、手続きが終了した

後に呼ばれた場所に戻るために戻り番地を記憶しなければならない。これらの手続き呼び出し・実行に必要な記憶場所として、呼び出しのたびにスタック上に連続した領域を確保していく手法が、再帰手続きの実行時環境としては一般的である。この連続した領域をフレームと呼ぶ。手続きが終了した場合、確保された領域はスタック上から削除される。この順序は呼ばれた順番と常に逆であるため、スタックの利用が適している。

手続きの呼び出しのたびに局所変数の格納場所が異なるということは、コンパイル時に変数のアドレスが決定できないことを意味する。しかし、一回の手続き呼び出しの際にスタック上に確保される領域は同じ大きさであるため、その領域のある点からの相対的なアドレスは一定である。これを利用して、手続きを実行するために確保される領域の相対アドレスで局所変数をアクセスするような目的プログラムを生成すればよい。実行時には、手続きの呼び出しごとにベースレジスタが新しく確保された領域の決まった場所を指し、そのベースレジスタからの相対アドレスで変数をアクセスする。このために、ベースレジスタ相対でアドレスを決定する命令が、それぞれの演算命令、転送命令に存在する。

局所変数に対して、大域変数は複数の実体に対応することはないため、ある決まった領域を確保しておけばよい。

ここまで説明してきた実行時環境をまとめたものを図 20 に示す。この図は手続きがネストして 3 回呼ばれているところであり、最後の 2 つのフレームは再帰呼び出しによる同じ手続きのものである。スタックは 0 番地を底として大きい番地に延びていくので、メモリのイメージに従い一番上がスタックの底である。スタックの底には、大域変数の領域が確保されている。一つのフレームは、引数、戻り番地、動的リンク、局所変数の領域からなる。引数は呼び出し側が積み、call 命令によって戻り番地が自動的に積まれる。動的リンクは、手続きが終了して戻る場合にベースレジスタを元の値に戻すために使われる。

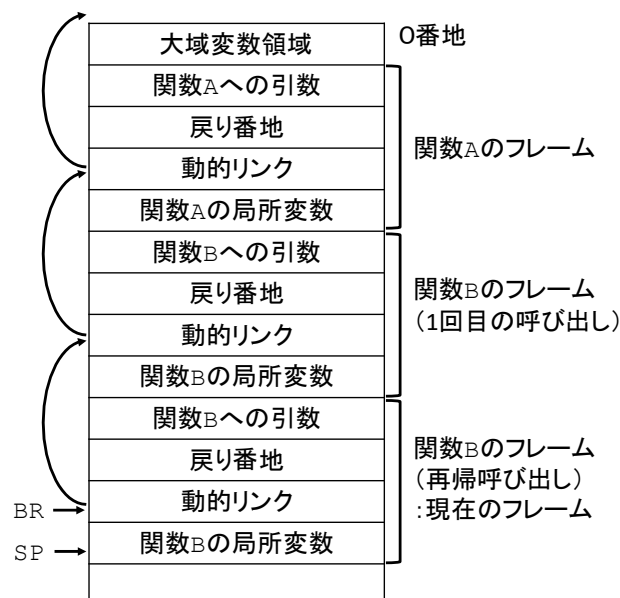


図 20 実行時環境

3.5.3 記号表

手続きを導入したことによって、記号表の扱いも異なるものとなる。手続きの名前からその

手続きのプログラムのアドレスを参照するために、変数と同じ記号表を用いる。変数の格納場所が手続きの開始アドレスに変わっただけである。また、変数に関しては大域変数と局所変数の区別を行う必要がある。手続き内の変数は、局所変数としてその手続きで定義されているか、または引数である場合は局所変数とみなされるが、そうでない場合は大域変数とみなされる。コンパイラでも 2 つの記号表を用意し、1 つを大域変数用、もう 1 つを局所変数用の記号表とする。手続きのコンパイルでは、変数のアドレスを求めるために、まず局所変数用の記号表を参照し、そこにはない場合は大域変数用の記号表を参照すればよい。

手続きの開始アドレスは大域変数用の記号表に入れなければならない。なぜなら、手続きの名前は原始プログラム全体で参照できるからである。

3.5.4 forward 文

図 21(a)のような原始プログラムをコンパイラが先頭から順番に処理する場合、**proc1** の目的プログラムが最初に出力される。しかし、**proc1** を処理している段階では、まだ **proc2** を処理していないために、**proc1** の中で呼ばれている **proc2** の開始アドレスが決定できない。また、コンパイラは **proc2** があることも知らないで、エラーとしてもよい。このために、前方参照をする名前は原始プログラムの先頭で、使うことを宣言しておく必要がある。例えば、**forward** 文を導入し、図 21(b)のように先頭で宣言するようにすれば、コンパイルの初期段階で前方参照をする全手続きを知ることができる。コンパイラは **forward** 文を読み込んだ段階で、開始アドレスを決定し、実際の本体にそのアドレスを使用すればよい。**forward** 文を使用せずとも、まだ宣言されていない手続きが呼び出されていることを発見した段階で、開始アドレスを仮定してもよい。

<pre> program forwardref; var i; ... procedure proc1(a); var j; begin ... proc2(j) ... end; procedure proc2(a); var k; ... begin ... end. </pre>	<pre> program forwardref; var i; ... forward proc2; procedure proc1(a); var j; begin ... proc2(j) ... end; procedure proc2(a); var k; ... begin ... end. </pre>
(a)	(b)

図 21 手続きの前方参照の例

4 実験報告書

実験の報告書には以下の内容を必ず含むこと。

- (1) 最終的に作成した記号表の構造と操作関数の仕様
- (2) `if` 文、`while` 文のコンパイルの仕様
- (3) 数式処理について「演算子順位構文解析」と「トップダウン構文解析による数式処理」のうち作成した方式の仕様（両方とも作成した場合は両方）
- (4) 演算子順位構文解析とトップダウン構文解析の比較検討
- (5) 手続きの実現の仕様
- (6) エラー処理について
- (7) 全体の構成概念図、実現した各関数の依存関係図
- (8) 作成したコンパイラが正しく動いているという証拠
- (9) 配布ファイル「`compiler2026.tar.gz`」に含まれる 3 つのプログラム(`hanoi.p`, `pi.p`, `nqueen.p`)の実行時間
- (10) その他、工夫した点、拡張した点について

5 参考文献

佐渡一広、寺島美昭、水野忠則：未来へつなぐデジタルシリーズ 24 コンパイラ

6 参考（平均的なプログラム実行時間）

<code>hanoi.p</code>	5,000～5,500
<code>nqueen.p</code>	23,000,000～25,000,000
<code>pi.p</code>	330,000～360,000